
Área Científica	Programação de Sistemas de Informação
Unidade Curricular	Introdução às Linguagens e Tecnologias Web
Ano Letivo / Semestre	2025/2026 – 2º Semestre
Docente Responsável	Prof. ^a Catarina Silva
Documento	Guião Prático Laboratorial 2
Tema	<i>Funções, DOM, Eventos, JSON e Fetch API</i>

Objetivos do Guião Prático

Este guião prático introduz a modularidade do código e a comunicação com serviços externos. O foco será na manipulação avançada da página e na integração de dados de forma assíncrona.

- Estruturar código em blocos lógicos e reutilizáveis (Funções).
- Manipular e criar elementos na árvore DOM em resposta a eventos de formulários.
- Compreender a estrutura de dados JSON e a sua conversão para objetos JavaScript.
- Efetuar pedidos de rede assíncronos (AJAX) sem recarregar a página através da Fetch API.

Estrutura Base do Projeto

Com a introdução da comunicação assíncrona (AJAX), o nosso código JavaScript passará a gerir não apenas as interações locais do utilizador, mas também as respostas de servidores externos. Abaixo apresenta-se o esqueleto recomendado para a organização do código.

Ficheiro HTML (index.html)

```
<!DOCTYPE html>
<html lang="pt">
<head>
  <meta charset="UTF-8">
  <title>Título do Exercício</title>
</head>
<body>
  <h1>Integração de Dados</h1>
  <button id="btnCarregar">Carregar Dados</button>
  <div id="resultado"></div>

  <script src="script.js"></script>
</body>
</html>
```

Ficheiro JavaScript (script.js)

```
// 1. Seleção de Elementos DOM
let btnCarregar = document.getElementById("btnCarregar");
let divResultado = document.getElementById("resultado");

// 2. Declaração de Funções (Lógica isolada)
function processarDados(dados) {
  divResultado.innerHTML = "<p>" + dados.nome + "</p>";
}

// 3. Event Listeners e Fetch API
btnCarregar.addEventListener("click", function() {

  // Efetua o pedido assíncrono
  fetch('https://api.exemplo.com/dados')
    .then(function(resposta) {
      return resposta.json(); // Converte texto para JS
    })
    .then(function(dadosJson) {
      processarDados(dadosJson); // Invoca a função
    })
    .catch(function(erro) {
      console.error("Erro no pedido:", erro);
    });
});
```

Exercício 1

Escreva uma função chamada `saudacaoPersonalizada` que receba dois parâmetros: o nome de uma pessoa e a hora atual (um número inteiro entre 0 e 23). A função deve retornar "Bom dia, [nome]!" se a hora for menor que 12, "Boa tarde, [nome]!" se for entre 12 e 19, e "Boa noite, [nome]!" caso contrário. Invoque a função e imprima o resultado na consola.

1. No ficheiro JS, declare a função utilizando a sintaxe `function saudacaoPersonalizada(nome, hora) {...}`.
2. Dentro da função, crie uma estrutura `if / else if / else` para avaliar o valor da variável `hora`.
3. Retorne a *String* correspondente utilizando a palavra-chave `return` (ex: `return "Bom dia, " + nome + "!"`).
4. Fora da função, invoque-a fornecendo os argumentos necessários e envolva a chamada num `console.log()` para visualizar o resultado.

Exercício 2

Crie um ficheiro HTML com um parágrafo que contenha o texto "O JavaScript é incrível!" e um botão com o texto "Mudar Estilo". Escreva um script que, ao clicar no botão, altere o tamanho da letra do parágrafo para 24px, mude a cor do texto para azul e altere a cor de fundo para cinzento claro.

1. No HTML, construa a estrutura solicitada, garantindo que o parágrafo e o botão possuem identificadores únicos (`id`).
2. No JS, selecione os dois elementos através de `document.getElementById()`.
3. Adicione um *Event Listener* do tipo "click" ao botão.
4. Na função do evento, modifique as propriedades CSS do parágrafo: `elemento.style.fontSize`, `elemento.style.color` e `elemento.style.backgroundColor`.

Exercício 3

Crie uma página com um campo de entrada de texto (`input type="text"`) e um elemento `span` vazio ao lado. Adicione um evento de teclado (`keyup`) ao campo de texto para que, à medida que o utilizador escreve, o texto exato que está a ser digitado seja espelhado e exibido em tempo real dentro do `span`.

1. No HTML, insira o `<input>` e o `<span>`, atribuindo-lhes `ids`.
2. No JS, selecione os elementos e adicione o escutador de eventos "keyup" ao campo de texto.
3. Dentro da função, extraia o valor introduzido acedendo à propriedade `.value` do `input`.
4. Atualize o `innerHTML` ou `innerText` do `span` com o texto extraído.

Exercício 4

Desenvolva um formulário para alteração de palavra-passe com dois campos: "Nova Palavra-passe" e "Confirmar Palavra-passe", e um botão de submissão. Ao submeter (prevenindo o comportamento padrão do browser), o script deve verificar se os dois campos são estritamente iguais e se têm mais de 6 caracteres. Se a validação falhar, injete um parágrafo vermelho abaixo do formulário com a indicação do erro. Se for bem-sucedida, mostre uma mensagem verde de "Alterada com sucesso" e limpe os campos.

1. Estructure o formulário no HTML, englobando os campos num `<form>` com um `id`.
2. No JS, escute o evento "submit" do formulário. Adicione o parâmetro `event` à função (ex: `function(event)`).
3. A primeira instrução OBRIGATÓRIA da função deve ser `event.preventDefault();`.
4. Leia o `.value` de ambos os campos e utilize uma estrutura `if` com os operadores `===` e `>`.
5. Atualize o DOM consoante o resultado, alterando a cor e o texto da mensagem de *feedback*.

Exercício 5

Considere o seguinte objeto JavaScript que representa um carro: `let carro = {marca: "Toyota", modelo: "Corolla", ano: 2020}`. Primeiro, converta este objeto numa *String* JSON e imprima-a na consola. De seguida, pegue nessa *String* gerada, converta-a de volta para um objeto JavaScript e altere o ano para 2024.

1. Declare o objeto `carro` no seu ficheiro JS.
2. Crie uma nova variável e atribua-lhe o resultado de `JSON.stringify(carro)`. Imprima esta variável na consola.
3. Noutra variável, efetue a operação inversa utilizando `JSON.parse()` sobre a *String* gerada.
4. Aceda à propriedade do novo objeto (`novoObjeto.ano = 2024`) e altere o seu valor.
5. Imprima o objeto final para confirmar a alteração de estado.

Exercício 6

Utilizando a Fetch API, faça um pedido GET ao endereço `https://jsonplaceholder.typicode.com/todos/1`. Este endereço devolve um objeto JSON com os dados de uma tarefa. Capture a resposta, converta-a para JavaScript e utilize um `console.log` para imprimir **apenas** o título (`title`) da tarefa na consola.

1. Execute a função `fetch()` passando o endereço URL como argumento (em formato de *String*).

2. Encadeie o primeiro `.then(function(resposta) {...})` para capturar a resposta do servidor e retorne `resposta.json()`.
3. Encadeie um segundo `.then(function(dados) {...})` que irá receber os dados já convertidos para objeto JS.
4. Dentro deste bloco, acesse a propriedade correspondente e imprima-a no terminal.

Exercício 7

Crie uma página que simule a navegação num fórum ou blog. Siga os seguintes passos:

- a. Ao carregar a página, faça um pedido Fetch a `https://jsonplaceholder.typicode.com/posts?_1` para obter apenas os primeiros 5 artigos.
- b. Converta o JSON recebido e utilize o DOM (`document.createElement`) para criar um botão para cada artigo, cujo texto seja o `title` da publicação.
- c. Adicione um evento de clique a cada um destes botões criados dinamicamente.
- d. Quando o utilizador clicar num botão, o seu script deve efetuar um **novo pedido Fetch** para ir buscar apenas os comentários desse artigo específico (`https://jsonplaceholder.typ` substituindo o `{ID}` pelo `id` do artigo clicado).
- e. Exiba os e-mails e o corpo (`body`) dos comentários recebidos numa lista abaixo dos botões, substituindo os comentários anteriores caso o utilizador clique num artigo diferente (`innerHTML =`).